

Reviewer Pack — Minimal Local Homology Bounds Resolution Proof Width

Entry ed27ce3e8e74 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 23:44:55 UTC

Minimal Local Homology Bounds Resolution Proof Width

Entry ID: ed27ce3e8e74 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 23:44:55 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Homology) **Field B** (complexity object): Boolean Function Complexity: Resolution Proof Complexity

Statement:

For every satisfiable 3-CNF formula F with m clauses on n variables, the minimal local homology rank of F is upper-bounded by a function $f(n)$ such that $\log_2 f(n) \leq t(F)$, where $t(F)$ is the minimum resolution refutation size for F .

Rationale (proposer's reasoning):

Local homology provides a geometric interpretation of the complexity of resolution proofs, offering a potential bridge between algebraic geometry and computational complexity. By relating it to resolution proof width, we could uncover new insights into the structure of proof systems.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8d39a927aa006ef9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture regarding minimal local homology bounds on resolution proof width, support is indicated by a correlation coefficient greater than 0.8 between $\log_2 f(n)$ and $t(F)$, across all generated instances, where no instance has $\log_2 f(n) > t(F)$. Falsification occurs if any instance yields $\log_2 f(n) > t^*(F)$ or the correlation coefficient is less than 0.8.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic geometry AND local homology AND resolution proof complexity
- Boolean function complexity AND resolution proof width AND minimal local homology rank -
minimum resolution refutation size AND upper bound AND algebraic geometry local homology

Top relevant hits considered: - [http://arxiv.org/abs/math/0302340v2] Pure homology of algebraic varieties - [http://arxiv.org/abs/2110.02458v3] Magnitude homology of graphs and discrete Morse theory on Asao-Izumihara complexes - [http://arxiv.org/abs/1512.04490v2] Free factorization algebras and homology of configuration spaces in algebraic geometry - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1110.0455v1] QBF-Based Boolean Function Bi-Decomposition - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/2404.04038v1] Refutability as Recursive as Provability - [http://arxiv.org/abs/2005.01608v2] Algorithms yield upper bounds in differential algebra

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_3cnf(n, clause_density):
    clauses = []
    for _ in range(int(clause_density * n)):
        variables = list(range(1, n + 1))
        random.shuffle(variables)
        clause = [variables[0], -variables[1]]
        if random.choice([True, False]):
            clause = [-x for x in clause]
        clauses.append(clause)
    return clauses

def resolution_refutation_size(F):
    # Simplified version of resolution refutation size calculation
    m = len(F)
    n = max(abs(x) for clause in F for x in clause)
    return m * n

def minimal_local_homology_rank(F):
    # Placeholder function for minimal local homology rank calculation
    # This is a dummy implementation and should be replaced with actual computation
```

```

    return random.randint(1, 10)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [10, 15, 20, 25]
    results = []

    for n in n_values:
        log_2_f_n = []
        t_F = []

        for _ in range(30):
            F = generate_3cnf(n, random.choice([0.5, 1, 2]))
            m = len(F)
            log_2_f_n.append(math.log2(minimal_local_homology_rank(F)))
            t_F.append(resolution_refutation_size(F))

        mean_log = sum(log_2_f_n) / len(log_2_f_n)
        mean_t_F = sum(t_F) / len(t_F)

        correlation_coefficient = sum((x - mean_log) * (y - mean_t_F) for x, y in zip(log_2_f_n, t_F)) / len(log_2_f_n)

        results.append({
            "n": n,
            "log_2_f_n": log_2_f_n,
            "t_F": t_F,
            "mean_log": mean_log,
            "mean_t_F": mean_t_F,
            "correlation_coefficient": correlation_coefficient
        })

    metric_name = "Correlation Coefficient"
    metric_value = sum(result["correlation_coefficient"] for result in results) / len(results)
    instances_tested = 30 * len(n_values)
    n_max = max(result["n"] for result in results)
    conjecture_holds = all(result["correlation_coefficient"] > 0.8 for result in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)

```

```

print(f"TRIAL: {{\\"seed\\": {seed}, ...run_trial output...}}")
results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\\"mapping_undefined\\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing *TRIAL:* lines)

7. Test stdout (last 2KB)

```

TRIAL: {"seed": 11, ...run_trial output...}
TRIAL: {"seed": 23, ...run_trial output...}
TRIAL: {"seed": 37, ...run_trial output...}
TRIAL: {"seed": 53, ...run_trial output...}
TRIAL: {"seed": 71, ...run_trial output...}
TRIAL: {"seed": 89, ...run_trial output...}
TRIAL: {"seed": 103, ...run_trial output...}
TRIAL: {"seed": 127, ...run_trial output...}
TRIAL: {"seed": 149, ...run_trial output...}
TRIAL: {"seed": 167, ...run_trial output...}
TRIAL: {"seed": 191, ...run_trial output...}
TRIAL: {"seed": 211, ...run_trial output...}
TRIAL: {"seed": 233, ...run_trial output...}
TRIAL: {"seed": 257, ...run_trial output...}
TRIAL: {"seed": 277, ...run_trial output...}
TRIAL: {"seed": 311, ...run_trial output...}
TRIAL: {"seed": 347, ...run_trial output...}
TRIAL: {"seed": 389, ...run_trial output...}
TRIAL: {"seed": 421, ...run_trial output...}
TRIAL: {"seed": 463, ...run_trial output...}
TRIAL: {"seed": 503, ...run_trial output...}
TRIAL: {"seed": 547, ...run_trial output...}
TRIAL: {"seed": 593, ...run_trial output...}
TRIAL: {"seed": 631, ...run_trial output...}
TRIAL: {"seed": 677, ...run_trial output...}
TRIAL: {"seed": 727, ...run_trial output...}
TRIAL: {"seed": 773, ...run_trial output...}
TRIAL: {"seed": 821, ...run_trial output...}
TRIAL: {"seed": 877, ...run_trial output...}
TRIAL: {"seed": 929, ...run_trial output...}
RESULT: INCONCLUSIVE reason=insufficient_evidence

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses a placeholder function for calculating the minimal local homology rank, which is replaced with random values. This does not provide a meaningful empirical test of the conjecture since it lacks an actual computation of the minimal local homology rank.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code uses a placeholder function for calculating the minimal local homology rank with random values, which does not provide a meaningful empirical test of the conjecture.
| next: Replace the placeholder function with an actual computation of the minimal local homology rank and retest to properly evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17256
2	preregistration	ollama_remote	glm4:latest	0	0	23466
3	novelty	ollama_remote	glm4:latest	0	0	15671
4	novelty	ollama_remote	glm4:latest	0	0	9801
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17230
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19719
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13987
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11703
9	critic	ollama_remote	glm4:latest	0	0	16312
10	judge	ollama_remote	glm4:latest	0	0	9283

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154429 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/ed27ce3e8e74.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/ed27ce3e8e74.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/ed27ce3e8e74.tar.gz* (if generated)